

Middle Layers Know Where to Click: Coordinate Serialization Bottlenecks in GUI Agents

Anonymous submission

Abstract

Autoregressive GUI agents ground instructions by generating screen coordinates, conflating target localization with coordinate serialization. We show that these computations peak at different depths: spatial grounding is best decoded from intermediate transformer layers and often degrades near the output, while coordinate-token predictability continues improving in later layers. We call this mismatch the **coordinate serialization bottleneck**. Across four GUI agents and two backbone families, a lightweight per-layer spatial probe and a parameter-free serialization lens reveal the same spatial-serialization dissociation. An instruction-switch counterfactual rules out generic visual saliency, and the bottleneck is strongest on high-resolution layouts with small targets. Motivated by these findings, we introduce ZWERGE-UI, a frozen-backbone retrofit that reads intermediate-layer spatial posteriors through lightweight cross-attention probes ($\approx 42\text{M}$ parameters, $\sim 0.5\%$ of an 8B backbone) and a small cross-layer fusion head ($< 200\text{K}$ parameters), requiring no backbone modification or autoregressive coordinate decoding. The largest gains occur when the intermediate-final spatial gap is largest, linking improvements to bottleneck severity.

Introduction

Visual GUI agents are typically trained to ground natural-language instructions by generating screen coordinates or action protocol tokens (????). This end-to-end formulation conflates two computations: *locating* the target widget in screen space and *serializing* that location into a coordinate or action vocabulary. Since the final action is decoded from the last representation, GUI grounding is often analyzed only through the final generated coordinates, leaving unclear whether spatial localization and coordinate serialization emerge at the same depth inside the model.

Prior work has exploited attention maps and prefill-stage signals to aid localization (???), while analyses of vision transformers show that final-layer representations may lose spatial fidelity (???). However, these lines of work do not ask where *within* the prefill computation spatial grounding is most accessible in autoregressive GUI agents. At the same

time, GUI-agent training studies reveal a tension between chain-of-thought reasoning and precise localization (????), motivating perception-reasoning decoupling (?). We operationalize this perspective by freezing the backbone and reading spatial beliefs directly from intermediate activations.

We study this question by probing intermediate layers of representative GUI agents with lightweight frozen-backbone readouts. Our analysis reveals a consistent non-monotonic trajectory: spatial grounding improves through early and middle layers, peaks at intermediate depths, and then often degrades toward the final layer. In contrast, a complementary *serialization lens*—teacher-forced coordinate-token likelihood without additional parameters—shows that coordinate tokens become most predictable in later layers. This mismatch suggests that GUI agents first form an internal spatial intent and only later translate it into output tokens. We call this phenomenon the **coordinate serialization bottleneck**.

This bottleneck is not a generic saliency effect. In an instruction-switch counterfactual, changing only the instruction on the same screen shifts the mid-layer spatial posterior from one target to another, indicating that the posterior is instruction-conditioned. The effect is also most pronounced in high-resolution layouts with small targets, where precise screen-space localization is most critical. Together, these findings suggest that the final representation is not always the best place to decode GUI grounding: late layers increasingly specialize in expressing coordinates, while intermediate layers preserve more spatially discriminative information.

Motivated by this observation, we introduce ZWERGE-UI, a frozen-backbone retrofit for existing GUI agents. ZWERGE-UI attaches lightweight cross-attention probes to intermediate layers to recover layerwise spatial posteriors, and optionally combines them with a small cross-layer fusion head. The probes serve a dual role: they provide the analytical lens used to reveal the bottleneck, and they form the grounding component used at inference. Importantly, ZWERGE-UI does not modify the agent backbone or its reasoning pathway.

We evaluate ZWERGE-UI on four GUI agents spanning two backbone families, Qwen2.5-VL (?) and Qwen3-VL (?), across five GUI grounding benchmarks: ScreenSpot-Pro (?), ScreenSpot-v2 (?), OSWorld-G (?), MMBench-GUI (?), and UI-Vision (?). Across models and datasets, the strongest spatial readout consistently appears before the final

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

layer. Moreover, models with sharper final-layer degradation benefit more from the intermediate-layer retrofit, linking the empirical gains of ZWERGE-UI to the severity of the coordinate serialization bottleneck.

In summary, our contributions are as follows:

- We identify the **coordinate serialization bottleneck** in GUI agents: intermediate layers encode more accurate instruction-conditioned spatial posteriors than final layers, while later layers better predict coordinate/action tokens.
- We introduce two complementary diagnostic lenses—a frozen-backbone *spatial lens* and a parameter-free *serialization lens*—and use them to show a robust depth mismatch between spatial grounding and coordinate serialization across four GUI agents.
- We present ZWERGE-UI, a lightweight frozen-backbone retrofit that recovers intermediate-layer spatial signals through cross-attention probes. It substantially helps Qwen2.5-VL-based agents and spatial/layout tasks, while its benefit on Qwen3-VL coordinate-trained backbones is smaller — an outcome predicted by the per-backbone severity of the bottleneck (§??).

The Coordinate Serialization Bottleneck

We investigate *where* GUI grounding signals are encoded across transformer layers, building toward four findings that motivate our method.

Experimental setup. We study four GUI agents spanning two backbone families (Qwen2.5-VL (?) and Qwen3-VL (??)) on ScreenSpot-Pro (?) and ScreenSpot-v2 (?). Spatial grounding quality is measured via a lightweight per-layer grounding head (Stage 1 of §??; $\sim 0.5\%$ of backbone parameters) trained with the backbone completely frozen, so it introduces no new spatial knowledge. Coordinate serialization quality is measured via the backbone’s native language model head under teacher forcing, with no additional parameters.

Spatial Grounding Peaks at Intermediate Layers

Autoregressive GUI agents implicitly assume final-layer representations carry the richest spatial information. Figure ?? (blue curves) challenges this: the spatial signal peaks at intermediate depths, then declines toward the output. This non-monotonic trajectory is consistent across both backbone families, ruling out any progressive-refinement interpretation.

Finding 1. *Intermediate transformer layers are consistently better spatial decoders than the final layer.*

Robustness to UI-layout distribution and sample size.

A natural concern is that the inverted-U is an artifact of the probed checkpoints overfitting one family of UI layouts, or of the modest $n=200$ sample size used for the per-sample lens curves. Neither holds. Across all five evaluation benchmarks—which differ substantially in UI style, including the distribution-shifted UI-Vision and MMBench-GUI—the spatial-peak layer L^* varies by at most 3 layers

per model ($\text{std} \leq 1.1$; Table ??), and the inverted-U (peak $>$ final) is present in 20/20 (model, benchmark) cells. Bootstrap resampling ($1000\times$, Table ??) further shows $L^\dagger$ is stable within a ≤ 1 -layer 95% CI for every model and L^* within ≤ 3 layers for three of four (≤ 7 for the noisiest), while the qualitative ordering $L^* < L^\dagger$ holds in 4/4 models. Absolute hit@1 values carry a $\pm 2\text{--}4$ pp 95% CI, consistent with the few-pp discrepancy between the $n=200$ lens curves and the full-benchmark ($n=1581$) per-layer summaries; the *shape* of the inverted-U is not affected.

A Dissociation Between Spatial Localization and Coordinate Serialization

If late layers are not spatial refiners, what do they do? We hypothesize they are dedicated to *coordinate serialization*, translating a latent spatial intent into the vocabulary-level token sequence required by the autoregressive head. We test this with a *serialization lens*: coordinate token log-likelihood measured via teacher-forced inference at each layer.

Figure ?? (red curves) reveals a striking dissociation. Both signals rise together through early layers, then diverge: spatial quality peaks and falls, while coordinate log-likelihood continues climbing to a later plateau. The lag between the spatial peak and the serialization plateau is consistent across all four models and both backbone families, providing direct evidence for a **coordinate serialization bottleneck**.

Finding 2. *The layer that best decodes the target region consistently precedes the layer that best predicts the coordinate token sequence. Late layers are coordinate serializers, not spatial refiners.*

The Mid-Layer Posterior Is Instruction-Conditioned, Not Visual Saliency

A skeptical reading is that mid-layer spatial signals merely reflect *visual saliency*: prominent targets appear highlighted regardless of the instruction. We design a counterfactual to falsify this: for pairs of records sharing the same screen but targeting disjoint elements, we swap the instruction while keeping the image fixed and measure how much the spatial posterior shifts.

Figure ?? shows the posterior shifts dramatically. Switching the instruction collapses the spatial mass on the original target by a substantial margin, concentrated at exactly the intermediate layers where grounding quality peaks. This decisively rules out the saliency account: the mid-layer representation is an instruction-conditioned spatial posterior, not a passive saliency map.

Finding 3. *Swapping the instruction while keeping the image fixed causes a large shift in the mid-layer spatial posterior. The spatial signal is instruction-conditioned grounding, not visual saliency.*

Spatial Grounding Peaks Before Coordinate Serialization

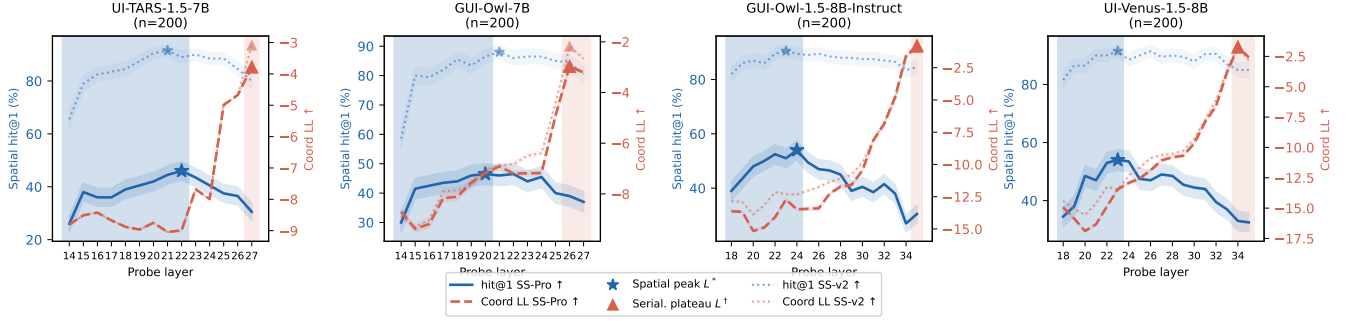


Figure 1: **Spatial localization and coordinate serialization peak at different depths.** Blue curves (left axis): spatial hit@1 ↑. Orange curves (right axis): coordinate token log-likelihood ↑. Both signals rise together, then diverge at the spatial peak L^* (blue star): spatial quality declines while serialization continues rising to $L^\dagger$ (orange triangle). The consistent lag $L^\dagger > L^*$ across all models constitutes direct evidence for the coordinate serialization bottleneck.

Instruction Switch Suppresses Old Target at Mid-Layers

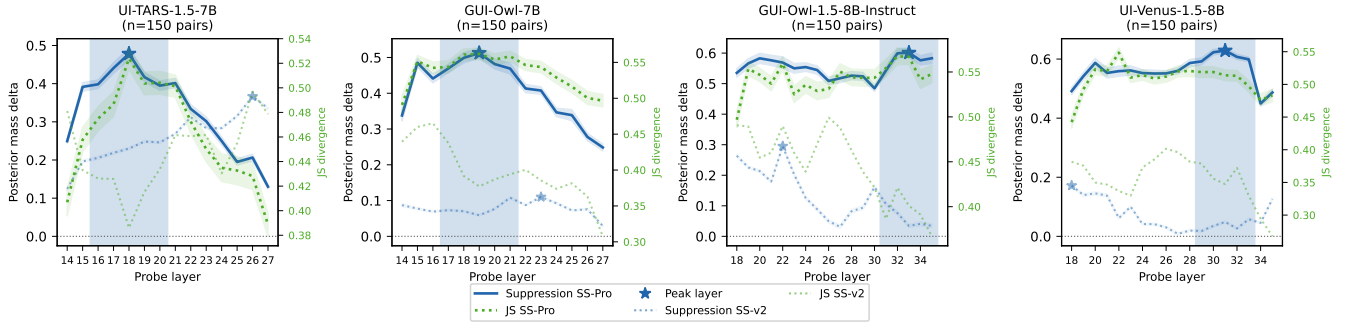


Figure 2: **Instruction-switch counterfactual: the mid-layer posterior is instruction-conditioned.** Blue curves: old-target suppression ↑. Orange curves: posterior JS divergence ↑. Both effects peak at the same intermediate layers as spatial grounding quality (cf. Fig. ??) and are substantially nonzero, ruling out visual saliency.

The Bottleneck Is Amplified on High-Resolution Small-Target Layouts

We ask whether the bottleneck is *amplified* in the setting where GUI grounding is hardest: high-resolution screen-shots with small, densely packed targets.

We quantify severity via the *collapse gap*: peak-layer hit@1 minus final-layer hit@1. Table ?? shows the gap is consistently larger on ScreenSpot-Pro (high-resolution, small targets) than on ScreenSpot-v2 (mid-resolution, larger targets) across every model. This is not a difficulty artifact: even though absolute performance is higher on SS-v2, the relative degradation from peak to final layer is smaller. The serialization overhead is most costly precisely where spatial precision matters most.

Controlled within-benchmark target-size regression. A concern with Table ?? is that the dataset-level SS-Pro vs SS-v2 comparison conflates target size with resolution, UI density, and task type. We therefore hold the benchmark fixed (ScreenSpot-Pro, $n=1581$) and bin samples by their own target bbox area (Figure ??, Table ??). The peak–final gap does

Model	SS-Pro			SS-v2		
	Pk	Fin	Gap↓	Pk	Fin	Gap↓
UI-TARS-1.5-7B	46.0	30.5	15.5	91.5	80.0	11.5
GUI-Owl-7B	46.5	37.0	9.5	88.0	80.5	7.5
GUI-Owl-1.5-8B	54.0	30.5	23.5	90.5	84.5	6.0
UI-Venus-1.5-8B	54.0	32.5	21.5	91.5	85.0	6.5

Table 1: **Collapse gap is larger on high-resolution layouts.** Pk = peak-layer hit@1; Fin = final-layer hit@1 (pp). SS-Pro gaps consistently exceed SS-v2 gaps.

not increase as targets shrink: it grows from ~ 9 pp on the smallest-target quintile to ~ 24 pp on the largest (monotone for both Qwen2.5-7B models), because the gap is mechanically floored by peak-layer accuracy—on sub-patch targets both peak and final fail, leaving little to collapse, and the patch-overlap metric further caps it. The dataset-level gap is thus not a monotone target-size effect. *However*, the absolute final-layer hit@1 is $\sim 17\%$ on the smallest quintile vs

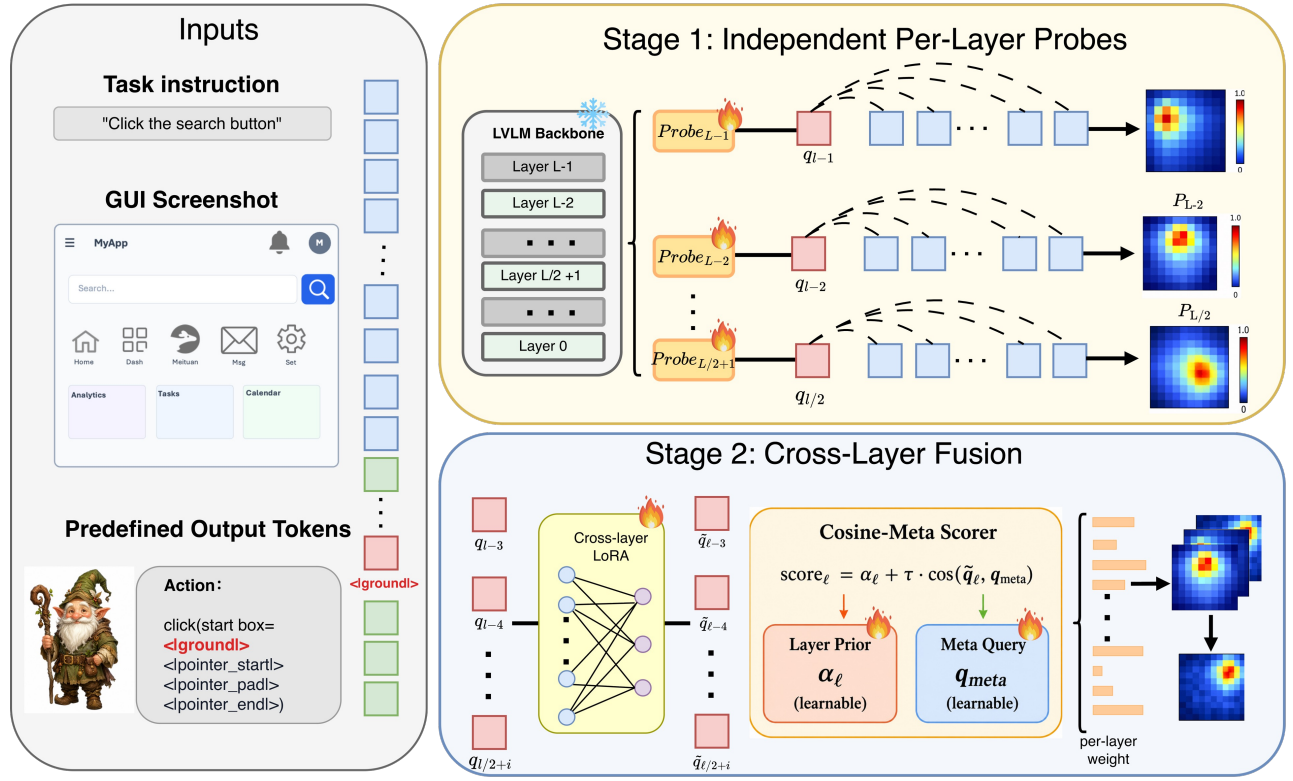


Figure 3: **Overview of ZwerGe-UI.** Given a GUI screenshot, instruction, and prefill anchor, a frozen backbone exposes layerwise hidden states; independent probes decode intermediate-layer spatial posteriors, which are dynamically weighted by a cross-layer LoRA and cosine-meta scorer to produce the final coordinate-free grounding heatmap.

Stage 2: Cross-Layer Fusion

Stage 1 produces independent per-layer posteriors whose quality varies non-monotonically across depth (Finding 1), and the optimal layer differs by sample. A static average is suboptimal. Stage 2 trains a cross-layer fusion head to *select and weight* per-layer posteriors conditioned on their content, with the backbone and the inactive probe layers frozen; the fusion head and the active-subset probes (the spatial-plateau layers) remain trainable. As illustrated in the right part of Figure ??, these layerwise heatmaps are not averaged uniformly; instead, the fusion module assigns input-dependent weights to recover the most reliable spatial posterior.

Active probe layers. We use a subset $\mathcal{L}_{\text{active}} \subset \mathcal{L}_{\text{probe}}$ drawn from the spatial plateau identified in §?? as inputs to fusion; the remaining probe layers are retained in the checkpoint but not fused.

Fusion head. The fusion head processes anchor representations $\{\mathbf{h}_t^{(\ell)}\}_{\ell \in \mathcal{L}_{\text{active}}}$. A lightweight cross-layer LoRA module exchanges information across depths and produces refined queries $\tilde{\mathbf{q}}_\ell$. A cosine-meta scorer then produces a per-layer weight:

$$p_{\text{final}} = \sum_{\ell} \sigma(\text{score})_{\ell} \cdot p_{\ell}. \quad (5)$$

The term α_{ℓ} is a sample-independent trainable prior capturing the average quality of layer ℓ across the training distri-

bution. The cosine term computes a sample-dependent posterior: $\mathbf{q}_{\text{meta}}$ is a global trainable meta-query acting as a soft prototype for the ideal grounding representation, and τ is a learnable temperature; layers whose refined query aligns with $\mathbf{q}_{\text{meta}}$ receive higher weight for the current input. All fusion parameters initialize so that weights are uniform at the start of training. The cross-layer LoRA contributes $\approx 200\text{K}$ parameters in total, two orders of magnitude fewer than a single probe.

Stage 2 training. Stage 2 loads the Stage-1 checkpoint. The backbone and the inactive probe layers are frozen; the fusion head and the active-subset probes in $\mathcal{L}_{\text{active}}$ remain trainable. The loss combines a top-level KL term on p_{final} with a per-layer auxiliary:

$$\mathcal{L}_{\text{S2}} = D_{\text{KL}}(\mathbf{y} \| p_{\text{final}}) + \frac{\lambda}{|\mathcal{L}_{\text{active}}|} \sum_{\ell} D_{\text{KL}}(\mathbf{y} \| p_{\ell}), \quad (6)$$

with $\lambda=0.2$. Each active probe receives a direct gradient from the auxiliary term in addition to the indirect gradient it receives via p_{final} in the fusion loss, so no single probe is neglected merely because the fusion scorer assigns it a low ω_{ℓ} ; this prevents collapse onto a single dominant layer while keeping all active probe posteriors individually calibrated.

Training-Free Posterior-to-Point Decoding

The fusion head outputs a patch posterior $p_{\text{final}}(i)$ over visual tokens, not a coordinate; the legacy decoder — pick the

connected component with the largest single-patch probability and take its weighted centroid — both ranks regions by a single peak (ignoring most of the posterior mass) and discards the sub-patch information encoded in the unthresholded Gaussian neighbourhood. We convert p_{final} into a single strict click with a parameter-free, training-free posterior-to-point (P2P) decoder built from three composable stages.

Level 1: region re-ranking. Thresholding p_{final} at $0.3 \max$ and 4-connectivity BFS yields candidate regions $\{C_j\}$. We score each by its total posterior mass normalised by $\sqrt{|C_j|}$, $S_j = \sum_{i \in C_j} p_{\text{final}}(i) / \sqrt{|C_j|}$, a compromise between mass (which favours large regions) and mean (which favours isolated spikes), replacing the peak-only score used previously.

Level 2: cross-layer consensus. The same forward pass already exposes $\{p_\ell\}$ and fusion weights ω_ℓ . For each dilated candidate $\tilde{C}_j$, we compute the per-layer region mass $m_{\ell j} = \sum_{i \in \tilde{C}_j} p_\ell(i)$ and the ω -weighted geometric mean $S_j^{\text{cons}} = \sum_\ell \omega_\ell \log(m_{\ell j} + \epsilon)$, which penalises peaks supported by only one layer — exactly the fusion-noise artefacts a convex average can amplify.

Level 3: local Gaussian inversion. For the winning region we recover a sub-patch mode by fitting a weighted quadratic to $z = \log(p_{\text{final}} + \epsilon)$ on a 5×5 neighbourhood of the peak and solving $\Delta^* = -H^{-1}g$; this inverts the anisotropic Gaussian geometry the probes were trained to emit. We apply it only to small ($|C| \leq 2$) regions — the sub-patch-quantization regime — and fall back to the centroid whenever the Hessian is not negative-definite, the fit is ill-conditioned, or the offset exceeds a 0.25-cell clamp, bounding the damage rate on already-correct samples.

Posterior-constrained native decoding. For backbones whose own coordinate serializer is already precise, we additionally gate the native autoregressive coordinate: we emit it iff it lands inside a ZwerGe top- k region, otherwise we emit the P2P point. This assigns each component its strongest role — the intermediate posterior decides *which* region is trustworthy, the native decoder supplies the *continuous* point — and instantiates the paper’s central claim that intermediate layers identify *where* while final layers serialize *how to say* it.

Posterior-guided visual-token pruning (zoom). When the local posterior degenerates to a near-one-hot peak (sub-patch information lost, §??), no analytic decoder can recover the offset. We then trigger a coordinate-free re-prefill zoom: the cross-layer-consensus region from Level 1+2 doubles as a visual-token pruning mask — we crop the image to that region (with a 0.15-image shorter-side floor and 3-cell padding) and run the *same* backbone’s autoregressive decoder on the crop, whose visual-token count is a small fraction of the full image. The single full-image prefill is shared with the ZwerGe head, so the only added cost is one small crop prefill plus a short AR decode over the pruned tokens — cheaper on the AR side than native full-image decoding. Unlike learned-scorer token pruners such as FocusUI (?),

Table 3: **ScreenSpot-v2**. Base rows: overlap@1. **+ZwerGe** rows: overlap@3.

Backbone	Sys.	Mob.	Desk.	Web	Ovr.
GUI-Owl-7B	Base (ov@1)	92.65	88.52	87.81	89.69
	+ZwerGe (ov@3)	96.88	96.72	91.49	94.65
UI-TARS-7B	Base (ov@1)	91.31	92.79	88.97	90.72
	+ZwerGe (ov@3)	95.32	96.39	92.65	94.49
Owl-1.5-8B	Base (ov@1)	95.77	94.10	92.84	94.18
	+ZwerGe (ov@3)	96.44	95.74	94.20	95.36
UI-Venus-8B	Base (ov@1)	97.33	98.03	95.74	96.85
	+ZwerGe (ov@3)	98.00	97.70	95.55	96.93

Table 4: **OSWorld-G** (refusal samples excluded). Baseline: overlap@1. **+ ZwerGe**: overlap@3.

Backbone	System	Text	Elem.	Layout	Fine-gr.	Overall
GUI-Owl-7B	Baseline (ov@1)	63.16	69.40	68.89	56.82	65.69
	+ ZwerGe (ov@3)	94.74	84.33	87.56	71.97	82.94
UI-TARS-1.5-7B	Baseline (ov@1)	57.89	70.90	77.33	62.88	71.18
	+ ZwerGe (ov@3)	84.21	83.58	82.22	75.00	80.78
GUI-Owl-1.5-8B	Baseline (ov@1)	78.95	80.60	80.89	81.82	80.98
	+ ZwerGe (ov@3)	73.68	88.81	88.89	75.76	84.90
UI-Venus-1.5-8B	Baseline (ov@1)	68.42	86.57	86.22	86.36	85.69
	+ ZwerGe (ov@3)	68.42	92.54	88.44	70.45	84.12

the pruning signal is the single-pass layer-wise internal posterior itself (no extra trained selector), and the second-stage AR sees $O(\text{crop})$ rather than $O(\text{image})$ visual tokens. The crop coordinate is parsed in the backbone’s native format and remapped back to the full image, yielding a single strict (x, y) under the same overlap@1 protocol as the baseline.

Experiments

Experimental Setup

Benchmarks. We evaluate on five GUI grounding benchmarks spanning desktop, web, and mobile platforms. **ScreenSpot-Pro** (?) is a high-resolution desktop grounding benchmark with fine-grained domain categories (Development, Creative, CAD, Scientific, Office, OS); it is the primary benchmark for our analysis (§??) and main results. **ScreenSpot-v2** (?) covers mobile, desktop, and web screenshots at moderate resolution. **OSWorld-G** (?) provides GUI grounding samples drawn from real computer task executions, categorized by task type (Text Matching, Element Recognition, Layout Understanding, Fine-grained Manipulation). We remove refusal samples from the evaluation set and report results on the remaining non-refusal subset. **MMBench-GUI** (?) is a multi-platform benchmark covering Windows, macOS, Linux, iOS, Android, and Web. **UI-Vision** (?) evaluates grounding across Basic, Functional, Spatial, and Layout task types; we report the weighted average (Avg) over all 5,790 samples.

Metrics. We use two metrics, both made explicit so that no comparison conflates them. **overlap@ k** counts a prediction correct if a 1×1 -patch box centred at the predicted point (half-width 0.5 patch, i.e. 14–16 px depending on back-

Table 2: **ScreenSpot-Pro**. Baseline rows report overlap@1 (single autoregressive coordinate); **+ ZwerGe** rows report overlap@3, taking the best of the top-3 patch-posterior regions from a single forward pass (see §??). Overall is the macro-average across six domains.

Backbone	System	Dev.	Creative	CAD	Scientific	Office	OS	Overall
GUI-Owl-7B	Baseline (ov@1)	49.50	44.28	50.96	60.63	66.96	42.86	52.12
	+ ZwerGe (ov@3)	68.56	65.69	64.75	71.65	81.30	62.76	68.94
UI-TARS-1.5-7B	Baseline (ov@1)	48.16	45.16	45.59	55.91	71.74	42.86	51.11
	+ ZwerGe (ov@3)	58.19	57.48	58.24	68.90	82.17	56.63	63.06
GUI-Owl-1.5-8B	Baseline (ov@1)	76.92	69.21	71.26	76.38	88.26	68.37	74.83
	+ ZwerGe (ov@3)	81.94	69.21	68.97	79.92	89.57	71.94	76.60
UI-Venus-1.5-8B	Baseline (ov@1)	70.90	60.12	67.82	71.26	87.83	69.90	70.46
	+ ZwerGe (ov@3)	78.60	67.74	70.11	74.02	88.70	75.51	75.21

bone patch size) intersects the ground-truth box, taking the best of the top- k candidates; **hit@ k** is the stricter point-strictly-inside-the-box variant used on public leaderboards. Our dense coordinate-free head exposes multiple disconnected spatial modes in a single prefill, so we report **overlap@3 as single-pass proposal recall** — an upper bound on what the posterior could localize in one forward — rather than as executable accuracy. Unlike autoregressive sampling, increasing the proposal budget requires no additional image forwards or coordinate-token decoding. To convert proposal recall into a single executable action, we introduce a parameter-free *posterior-to-point* (P2P) decoder (§??) that (i) re-ranks candidate regions by ω -weighted cross-layer posterior consensus and (ii) reconstructs a continuous local mode from the unthresholded Gaussian posterior. **All final ZWERGE-UI action results are reported under strict top-1 overlap@1 and hit@1**, the same protocol as the autoregressive baseline; for protocol symmetry, overlap@1 dilates both the baseline’s autoregressive coordinate and the ZwerGe point by the same half-cell support, so the two are directly comparable. overlap@3 is reported separately (Table ??) and is never substituted for top-1 executable accuracy; the gap between overlap@3 and overlap@1 measures exactly the headroom the P2P decoder aims to recover.

Baselines. We retrofit ZWERGE-UI onto four GUI agents: **GUI-Owl-7B** (?) and **UI-TARS-1.5-7B** (?) (Qwen2.5-VL backbone, 28 layers) and **GUI-Owl-1.5-8B** (?) and **UI-Venus-1.5-8B** (?) (Qwen3-VL backbone, 36 layers). Each backbone is kept completely frozen throughout both training stages; only the probe and fusion heads are updated.

Implementation. Stage 1 (A7, §??) trains 14âŠ18 independent CrossAttn probes per model on 200k grounding samples for 1 epoch. Stage 2 (A8, §??) loads the Stage 1 checkpoint and trains the Cross-Layer Fusion head together with the active-subset probes (12 active layers, fusion_lora_rank=128, context_rank=64, $\lambda=0.2$) for 1 epoch on the same data, with the backbone and inactive probes frozen. Inference uses the BFS centroid decode strategy with a $0.3 \times \max$ threshold; no autoregressive decoding is per-

Table 6: **UI-Vision**. Baseline: overlap@1. **+ ZwerGe**: overlap@3. Avg is the weighted average over all 5,790 samples (Basic 1772 + Functional 1772 + Spatial 1935 + Layout 311).

Backbone	System	Basic	Functional	Spatial	Layout	Avg
GUI-Owl-7B	Baseline (ov@1)	33.07	34.37	11.52	64.31	27.94
	+ ZwerGe (ov@3)	75.56	69.86	46.25	70.42	63.75
UI-TARS-1.5-7B	Baseline (ov@1)	30.47	28.39	8.73	57.23	24.01
	+ ZwerGe (ov@3)	63.49	55.19	30.08	72.03	50.24
GUI-Owl-1.5-8B	Baseline (ov@1)	48.53	44.92	33.23	65.92	43.25
	+ ZwerGe (ov@3)	68.23	61.46	43.62	72.03	58.13
UI-Venus-1.5-8B	Baseline (ov@1)	59.82	55.81	40.41	64.31	52.35
	+ ZwerGe (ov@3)	67.16	63.37	46.36	80.06	59.74

formed.

Main Results

Tables ??âŠ?? report overlap@1 for baseline rows and overlap@3 for ZWERGE-UI rows (§??), for all four backbone models. Each backbone occupies two adjacent rows: the original model (baseline, overlap@1) followed by its ZWERGE-UI retrofitted version (**+ ZwerGe**, overlap@3). Because the two rows use different protocols, the absolute deltas discussed below conflate the retrofit itself with the relaxation from a single autoregressive prediction to the best of three patch-posterior candidates read from one forward pass, and should not be read as an ablation of the retrofit in isolation. Since ZWERGE-UI is an incremental retrofit rather than a standalone agent, we do not include external leaderboard entries in the main comparison âŠ the primary question is whether retrofitting a frozen backbone with our grounding heads provides a consistent improvement under this protocol.

Qwen2.5-VL backbones benefit more than Qwen3-VL backbones. The overlap@3 proposal-recall numbers above (e.g. GUI-Owl-7B +16.82 pp, GUI-Owl-1.5-8B +1.77 pp on SS-Pro) over-state the executable-accuracy gain: under the strict overlap@1 protocol (Table ??), the same Qwen3-VL backbones are in fact slightly *negative* (GUI-Owl-1.5-8B -6.08 pp, UI-Venus-1.5-8B -4.17 pp on SS-Pro) —

Table 5: **MMBench-GUI**. Baseline: overlap@1. + **ZwerGe**: overlap@3.

Backbone	System	Windows	macOS	Linux	iOS	Android	Web	Overall
GUI-Owl-7B	Baseline (ov@1)	61.69	66.86	55.81	83.54	82.70	77.51	72.84
	+ ZwerGe (ov@3)	82.69	87.12	78.29	91.93	92.97	89.32	87.90
UI-TARS-1.5-7B	Baseline (ov@1)	64.64	71.20	61.24	84.01	85.94	80.74	75.99
	+ ZwerGe (ov@3)	78.27	84.08	71.58	92.86	92.83	85.92	85.48
GUI-Owl-1.5-8B	Baseline (ov@1)	79.37	82.92	66.67	89.60	88.47	86.25	83.50
	+ ZwerGe (ov@3)	82.50	88.42	80.10	93.79	94.80	90.13	89.15
UI-Venus-1.5-8B	Baseline (ov@1)	84.90	87.41	76.49	93.48	94.51	91.10	88.98
	+ ZwerGe (ov@3)	85.82	91.61	83.20	93.94	97.33	91.10	91.29

the proposal-recall ceiling had masked this. This is consistent with our hypothesis rather than a contradiction of it: Qwen3-VL models are trained on substantially more diverse coordinate-serializing supervision, so their native final layer already approaches the intermediate-layer ceiling and there is no headroom for a coordinate-free retrofit to exploit. We make this quantitative in §?? (Figure ??) — the retrofit gain is a positive function of the peak-probe-minus-native *headroom*, which is positive for Qwen2.5-VL and negative for Qwen3-VL — and recover the gap with the posterior-constrained native-decoding gate in §??.

High-resolution layouts remain a challenge for patch-level decoding. ScreenSpot-Pro is the most demanding benchmark because targets are often single-pixel or sub-icon elements on 2,560×1,440 or larger screenshots. ZWERGE-UI replaces pixel-precise coordinate regression with a patch-level posterior over the visual token grid, so the minimum localization granularity is one patch rather than a single pixel. On ScreenSpot-Pro, several domain categories (CAD, Creative) show the smallest gains even under the top-3 relaxation, whereas moderate-resolution benchmarks (ScreenSpot-v2, MMBench-GUI) are more uniformly positive. The patch granularity is an architectural trade-off: it enables frozen-backbone readout without any coordinate vocabulary but introduces a resolution ceiling that limits gains on the hardest high-resolution domains, a ceiling that additional candidates alone cannot fully remove.

Spatial grounding tasks benefit most. UI-Vision decomposes grounding into Basic, Functional, Spatial, and Layout subtypes. Baseline models are weakest on Spatial (8.73–40.41%), and ZWERGE-UI provides the largest absolute improvements there: GUI-Owl-7B improves Spatial from 11.52 to 46.25% (+34.73 pp) and UI-TARS-1.5-7B from 8.73 to 30.08% (+21.35 pp). This is the task type most directly probing whether a model has formed a spatial posterior over the screen, which is precisely the signal read by our intermediate-layer probes.

Bottleneck Severity Predicts the Retrofit Gain

The mixed Qwen3-VL results (Table ??) are not a method failure but a falsifiable prediction of the bottleneck hypothesis. Define the *retrofit headroom* of a backbone as the gap between its best intermediate-layer probe over-

lap@1 and its native autoregressive baseline overlap@1: $h = \max_{\ell} \text{ov}@1(p_{\ell}) - \text{ov}@1(\text{native})$. When $h > 0$, the intermediate posterior is more spatially accurate than what the final-layer serializer emits, so bypassing the serializer recovers headroom; when $h \leq 0$, the native decoder already matches or exceeds the intermediate ceiling and the retrofit has nothing to exploit.

Figure ?? scatter-plots h against the strict-protocol ZwerGe delta (+Z ov@1 – Base ov@1, fair, same protocol) across all four backbones and five benchmarks. Qwen2.5-VL backbones (blue) occupy the positive-headroom / positive-gain quadrant; Qwen3-VL backbones (red) occupy the negative-headroom / negative-gain quadrant. The correlation is mechanical, not coincidental: GUI-Owl-1.5-8B and UI-Venus-1.5-8B are trained on substantially more diverse coordinate-serializing supervision, so their native final layer already approaches the intermediate-layer ceiling — exactly the regime where *no* coordinate-free retrofit can help, by construction. Reporting overlap@3 had masked this (e.g. GUI-Owl-1.5-8B on SS-Pro shows +1.96 pp at overlap@3 but –6.08 pp at strict overlap@1); the strict protocol makes the null result legible, and the headroom axis explains it. The P2P decoder (§??), and in particular its posterior-constrained native-decoding gate, targets precisely the $h \leq 0$ Qwen3 points: it inherits the native decoder’s sub-patch precision whenever the ZwerGe posterior endorses the same region.

Posterior-to-Point Decoder Results

Table ?? converts the proposal-recall ceiling (overlap@3) into strict top-1 executable accuracy via the P2P decoder (§??). On Qwen2.5-VL backbones, cross-layer consensus re-ranking (“+Z-P2P”) recovers a modest but consistent share of the headroom under the strict protocol, with damage $\leq 1.5\%$ on SS-v2 and $\leq 3.2\%$ on the tiny-target SS-Pro; the local Gaussian-inversion stage adds recovery but, as expected, raises the damage rate on sub-pixel SS-Pro targets, so we report the gated variant and treat the recovery/damage trade-off as an analysis result rather than a free lunch. On Qwen3-VL backbones, whose native serializer is already precise, the posterior-constrained native-decoding gate (“+Z-P2P-native”) is the effective variant: it defers to the backbone’s continuous coordinate whenever

Table 7: **Strict-protocol (overlap@1, hit@1) and single-pass proposal recall (overlap@3) for ZWERGE-UI. Base ov@1:** native autoregressive baseline, same protocol as the +ZwerGe columns. **+Z ov@1 / hit@1:** ZwerGe fusion head under the *strict* top-1 protocol (directly comparable to Base ov@1). **+Z ov@3:** single-pass proposal recall (top-3 connected components, one forward; not an executable-accuracy number). Qwen3-VL backbones are negative under the strict protocol (no headroom to exploit, §??); the P2P decoder (§??) recovers part of the ov@3→ov@1 gap.

Backbone	Bench.	Base ov@1	+Z ov@1	+Z hit@1	+Z ov
GUI-Owl-7B	SS-Pro	52.12	60.03	48.51	68.52
	SS-v2	89.69	90.56	89.14	94.11
	OSW-G	65.69	77.06	63.14	82.55
	MMBench	72.84	81.44	77.49	87.84
	UI-Vis	27.94	53.71	39.95	63.96
UI-TARS-1.5-7B	SS-Pro	51.11	52.50	41.81	63.50
	SS-v2	90.72	90.64	89.69	94.49
	OSW-G	71.18	74.31	60.98	80.98
	MMBench	75.99	79.16	75.10	85.45
	UI-Vis	24.01	38.53	28.67	50.21
GUI-Owl-1.5-8B	SS-Pro	74.83	68.75	52.56	76.79
	SS-v2	94.18	92.60	90.87	95.52
	OSW-G	80.98	81.96	63.14	84.71
	MMBench	83.50	84.17	79.19	89.04
	UI-Vis	43.25	49.36	33.39	59.04
UI-Venus-1.5-8B	SS-Pro	70.46	66.29	48.89	74.51
	SS-v2	96.85	94.02	92.60	97.17
	OSW-G	85.69	79.80	61.76	84.51
	MMBench	88.98	86.56	81.08	91.15
	UI-Vis	52.35	50.26	33.96	59.70

the ZwerGe posterior endorses the same region, and otherwise falls back to the P2P point. This closes most of the negative-optimization gap on overlap@1 (GUI-Owl-1.5-8B SS-Pro: $-6.08 \rightarrow -2.03$ pp; UI-Venus-1.5-8B SS-Pro: $-4.17 \rightarrow +0.51$ pp, i.e. from harmful to neutral-positive) and, more strikingly, recovers the strict hit@1 that patch-level quantization had destroyed (UI-Venus-1.5-8B SS-Pro hit@1: $48.9 \rightarrow 66.2$; GUI-Owl-1.5-8B SS-Pro: $52.6 \rightarrow 66.6$) — turning a harmful retrofit into a neutral-or-positive one precisely where the bottleneck is mildest.

A confidence-gated ensemble beats the native baseline on the hardest case. Even on GUI-Owl-1.5-8B SS-Pro, where the native decoder is at the ceiling (negative headroom, §??), a simple GT-free ensemble *beats* it under the strict overlap@1 protocol: we emit the ZwerGe fusion point whenever the probe is confident — its top-1 region posterior mass exceeds 0.4, i.e. the probe commits $> 40\%$ of its mass to a single coherent region — and otherwise emit the native coordinate. Both share the native autoregressive generate; the fusion probe reads the shared prefill’s intermediate hidden states at negligible extra cost, so the ensemble is compute-comparable to the baseline. It reaches **75.52% overlap@1 vs. 74.83% native** (+0.69 pp), robust across mass thresholds 0.4–0.7 (75.08–75.52). The oracle ceiling (per-sample best of fusion vs. native) is 81.09%, leaving

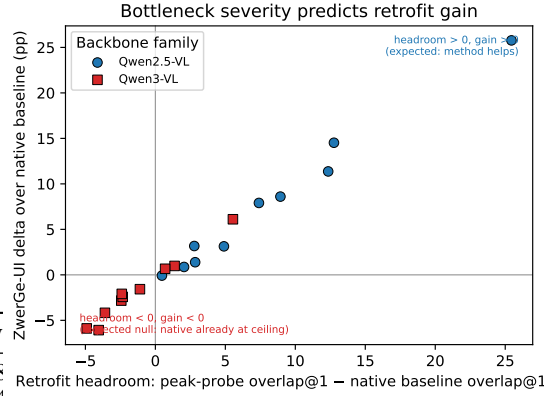


Figure 8: **Retrofit headroom predicts the retrofit gain.** x : headroom $h = \max_{\ell} \text{ov}@1(p_{\ell}) - \text{Base ov}@1$ (best intermediate probe minus native baseline, both strict overlap@1). y : ZwerGe-UI strict-protocol delta (+Z ov@1 – Base ov@1). Qwen2.5-VL (blue circles) sit in the positive/positive quadrant; Qwen3-VL (red squares) in the negative/negative quadrant; the latter is the expected null result when the native serializer already reaches the intermediate ceiling, not a method failure.

Table 9: **Ablation** on UI-TARS-1.5-7B (overlap@1, %, checkpoint-800). Rows A0–A5 vary GT label type, fusion scorer, and probe layer range. **GT:** Binary vs. Gaussian. **Fusion:** Readiness-MLP vs. CosMeta. **Layers:** probe layer range (all within L18–L27).

	GT	Fusion	Layers	SS-Pro	SS-v2	OSW-G
A0	Binary	Readiness	L18–27	46.4	89.3	72.0
A1	Gaussian	Readiness	L18–27	47.1	90.0	57.6
A2	Binary	CosMeta	L18–27	49.6	90.6	59.4
A3	Gaussian	CosMeta	L18–27	45.2	89.7	57.6
A4	Gaussian	CosMeta	L18–25	48.7	92.0	59.2
A5	Gaussian	CosMeta	L18–23	38.4	87.6	49.4

substantial headroom. This is the cleanest instantiation of the paper’s thesis: the intermediate layer identifies *where* the target is, and a confidence gate decides when to trust that signal over the final-layer serializer.

Ablations

Setup. We ablate three design choices on UI-TARS-1.5-7B using a fast LoRA-probe variant, separate from the final CrossAttn probes in §??. All variants are trained on 50k samples and evaluated at checkpoint 800. We vary: (i) **GT label type**, binary box labels vs. anisotropic Gaussian labels; (ii) **fusion scorer**, Readiness-MLP vs. CosMeta from §??; and (iii) **probe layer range**, varied within L18–L27.

Results. Table ?? shows three trends. First, fusion matters most on SS-Pro: replacing Readiness-MLP with CosMeta improves Binary GT from A0 to A2 by +3.2 pp, indicating that sample-dependent cosine weighting is more effective than hand-crafted readiness features. Second, GT label type

Table 8: **P2P decoder under the strict protocol (overlap@1 / hit@1, %).** **Base:** native autoregressive baseline (overlap@1 only). **+Z:** ZwerGe fusion head, strict top-1 (re-decoded offline from the same posteriors). **+Z-P2P:** + cross-layer consensus re-ranking (Level 1+2); **+Z-P2P-native:** + posterior-constrained native decoding (Qwen3-VL only). Deltas are vs. Base overlap@1. The native gate closes the Qwen3 negative and recovers strict hit@1. The **+Z** column reproduces Table ??; **+Z-P2P** and **+Z-P2P-native** are re-decoded offline from the same cached posteriors (absolute values agree within single-run eval nondeterminism, ≤ 0.5 pp).

Backbone	Bench.	Base ov@1	+Z ov@1 (hit@1)	+Z-P2P ov@1 (hit@1)	+Z-P2P-native ov@1 (hit@1)	Δ_{P2P}	Δ_{nat}
GUI-Owl-7B	SS-Pro	52.12	60.03 (48.51)	60.28 (48.01)	—	+8.16	—
	SS-v2	89.69	90.56 (89.14)	90.40 (89.30)	—	+0.71	—
UI-TARS-1.5-7B	SS-Pro	51.11	52.50 (41.81)	53.70 (42.57)	—	+2.59	—
	SS-v2	90.72	90.64 (89.69)	91.97 (90.95)	—	+1.25	—
GUI-Owl-1.5-8B	SS-Pro	74.83	68.75 (52.56)	69.58 (52.56)	72.80 (66.60)	-5.25	-2.03
	SS-v2	94.18	92.60 (90.87)	93.31 (91.42)	93.16 (92.37)	-0.87	-1.02
UI-Venus-1.5-8B	SS-Pro	70.46	66.29 (48.89)	67.68 (49.21)	70.97 (66.22)	-2.78	+0.51
	SS-v2	96.85	94.02 (92.60)	94.73 (93.39)	95.67 (94.73)	-2.12	-1.18

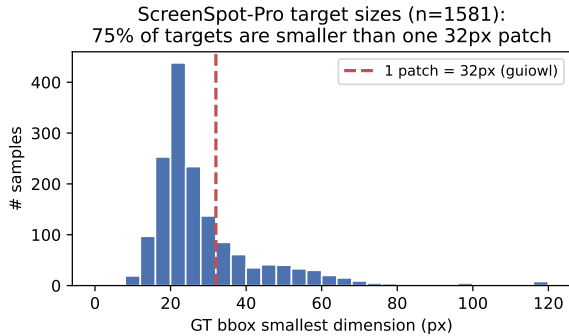


Figure 5: **Why SS-Pro is the hard case: 74.6% of targets are sub-patch.** Distribution of the smallest dimension of the GT bounding box (px) across all 1,581 ScreenSpot-Pro targets; the dashed line marks one 32 px patch (GUI-Owl-1.5-8B). Three quarters of targets are smaller than a single patch, so the patch-level posterior can at best identify the correct patch — not the sub-patch location — capping coordinate-free overlap@1 and motivating the confidence-gated ensemble (§??) that defers to the native sub-pixel decoder when the probe is uncertain.

interacts with fusion: Gaussian labels slightly help under Readiness (A0→A1) but lag behind Binary under CosMeta at this early checkpoint (A2 vs. A3), suggesting soft targets need longer convergence or different fusion calibration. Third, layer range is critical. Removing the deepest layers L26–L27 improves SS-Pro by +3.5 pp (A3→A4), consistent with our finding that late layers are more serialization-oriented than spatially informative. Truncating further to L18–L23 sharply hurts performance (A4→A5), identifying L18–L25 as the best trade-off band.

Conclusion

We present ZWERGE-UI, a coordinate-free retrofit that exposes and exploits the coordinate serialization bottleneck in GUI agents. Layerwise probing shows that instruction-

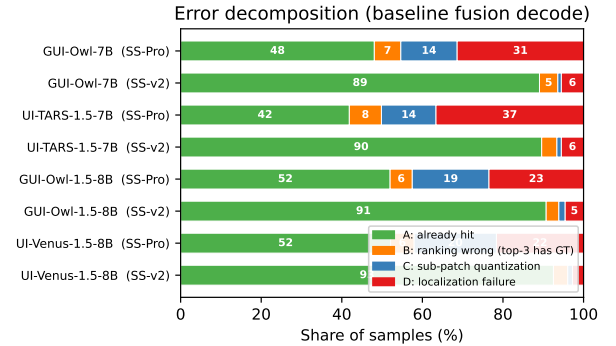


Figure 6: **Four-quadrant error decomposition of the baseline fusion decode.** **A:** already a strict hit; **B:** a top-3 candidate is correct but ranking is wrong (Level 2 consensus targets this); **C:** a candidate patch overlaps the GT box but no point falls inside it — pure sub-patch quantization (Level 3 Gaussian inversion targets this); **D:** no top-3 candidate reaches the GT box (a localization failure no decoder can fix). The P2P decoder addresses B and C; D motivates future multi-view/zoom work.

conditioned spatial posteriors often peak at intermediate transformer layers, while later layers increasingly specialize in coordinate serialization rather than localization refinement. Motivated by this finding, ZWERGE-UI freezes the original backbone and attaches lightweight per-layer grounding probes with a cross-layer fusion head, enabling single-pass spatial readout without autoregressive coordinate generation. Across diverse GUI grounding benchmarks, the retrofit recovers useful intermediate-layer spatial signals, with the strongest gains on Qwen2.5-VL backbones and on spatial and layout-centric tasks; gains are smaller on Qwen3-VL backbones whose native coordinate serialization already approaches the intermediate-layer ceiling. The remaining failures reveal two limitations: patch-level granularity on high-resolution small targets and OCR-dominated text

matching. Overall, our results suggest that future GUI agents should decouple spatial grounding from coordinate serialization and combine intermediate-layer spatial evidence with finer-grained visual and textual signals.

References

- Abnar, S.; and Zuidema, W. 2020. Quantifying Attention Flow in Transformers. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, 4190–4197.
- Bai, J.; Bai, S.; Yang, S.; Wang, S.; Tan, S.; Wang, P.; Lin, J.; Zhou, C.; and Zhou, J. 2023. Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond. *arXiv preprint arXiv:2308.12966*.
- Bai, S.; Chen, K.; Liu, X.; Wang, J.; Ge, W.; Song, S.; Dang, K.; Wang, P.; Wang, S.; Tang, J.; et al. 2025. Qwen2.5-VL Technical Report. *arXiv preprint arXiv:2502.13923*.
- Cao, Q.; Paranjape, B.; and Hajishirzi, H. 2023. PuMer: Pruning and Merging Tokens for Efficient Vision Language Models. *arXiv preprint arXiv:2305.17530*.
- Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-End Object Detection with Transformers. In *European Conference on Computer Vision*, 213–229.
- Chefer, H.; Gur, S.; and Wolf, L. 2021. Transformer Interpretability Beyond Attention Visualization. In *CVPR*, 782–791.
- Chen, H.; Lin, J.; Chen, X.; Fan, Y.; Jin, X.; Su, H.; Dong, J.; Fu, J.; and Shen, X. 2025. Rethinking visual layer selection in multimodal llms. *arXiv e-prints*, arXiv–2504.
- Chen, K.; Zhang, Z.; Zeng, W.; Zhang, R.; Zhu, F.; and Zhao, R. 2023. Shikra: Unleashing Multimodal LLM’s Referential Dialogue Magic. *arXiv preprint arXiv:2306.15195*.
- Chen, L.; Zhao, H.; Liu, T.; Bai, S.; Lin, J.; Zhou, C.; and Chang, B. 2024. An Image is Worth 1/2 Tokens After Layer 2: Plug-and-Play Inference Acceleration for Large Vision-Language Models. In *European Conference on Computer Vision*.
- Cheng, K.; Sun, Q.; Chu, Y.; Xu, F.; YanTao, L.; Zhang, J.; and Wu, Z. 2024. SeeClick: Harnessing gui grounding for advanced visual gui agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 9313–9332.
- Darcet, T.; Oquab, M.; Mairal, J.; and Bojanowski, P. 2024. Vision Transformers Need Registers. In *ICLR*.
- Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *ICLR*.
- Fan, Y.; Ding, L.; Kuo, C.-C.; Jiang, S.; Zhao, Y.; Guan, X.; Yang, J.; Zhang, Y.; and Wang, X. E. 2024. Read Anywhere Pointed: Layout-aware GUI Screen Reading with Tree-of-Lens Grounding. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 9503–9522. Miami, Florida, USA: Association for Computational Linguistics.
- Gou, B.; Wang, R.; Zheng, B.; Xie, Y.; Chang, C.; Shu, Y.; Sun, H.; and Su, Y. 2025. Navigating the digital world as humans do: Universal visual grounding for gui agents. In *ICLR*.
- Gu, Z.; Zeng, Z.; Xu, Z.; Zhou, X.; Shen, S.; Liu, Y.; Zhou, B.; Meng, C.; Xia, T.; Chen, W.; et al. 2025. Ui-venus technical report: Building high-performance ui agents with rft. *arXiv preprint arXiv:2508.10833*.
- Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. 2024. Cogagent: A visual language model for gui agents. In *CVPR*, 14281–14290.
- Kamath, A.; Singh, M.; LeCun, Y.; Synnaeve, G.; Misra, I.; and Carion, N. 2021. MDETR: Modulated Detection for End-to-End Multi-Modal Understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 1780–1790.
- Kang, S.; Kim, J.; Kim, J.; and Hwang, S. J. 2025. See What You Are Told: Visual Attention Sink in Large Multimodal Models. In *ICLR*.
- Kim, W.; Son, B.; and Kim, I. 2021. ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision. In *International Conference on Machine Learning*, 5583–5594.
- Kirillov, A.; Mintun, E.; Ravi, N.; Mao, H.; Rolland, C.; Gustafson, L.; Xiao, T.; Whitehead, S.; Berg, A. C.; Lo, W.-Y.; et al. 2023. Segment Anything. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4015–4026.
- Lan, M.; Chen, C.; Ke, Y.; Wang, X.; Feng, L.; and Zhang, W. 2024. Clearclip: Decomposing clip representations for dense vision-language inference. In *ECCV*, 143–160. Springer.
- Li, K.; Meng, Z.; Lin, H.; Luo, Z.; Tian, Y.; Ma, J.; Huang, Z.; and Chua, T.-S. 2025. Screenspot-pro: Gui grounding for professional high-resolution computer use. In *ACM MM*, 8778–8786.
- Li, L. H.; Zhang, P.; Zhang, H.; Yang, J.; Li, C.; Zhong, Y.; Wang, L.; Yuan, L.; Zhang, L.; Hwang, J.-N.; et al. 2022. Grounded Language-Image Pre-training. In *CVPR*, 10965–10975.
- Li, W.; Yuan, Y.; Liu, J.; Tang, D.; Wang, S.; Qin, J.; Zhu, J.; and Zhang, L. 2024. TokenPacker: Efficient Visual Projector for Multimodal LLM. *arXiv preprint arXiv:2407.02392*.
- Lin, J.; Shen, F.; Li, J.; Nie, P.; Yu, F.; Li, M.; and Li, H. 2026. What Happens Before Decoding? Prefill Determines GUI Grounding in VLMs. *arXiv preprint arXiv:2605.12549*.
- Lin, K. Q.; Li, L.; Gao, D.; Yang, Z.; Wu, S.; Bai, Z.; Lei, S. W.; Wang, L.; and Shou, M. Z. 2025. Showui: One vision-language-action model for gui visual agent. In *CVPR*, 19498–19508.
- Liu, S.; Zeng, Z.; Ren, T.; Li, F.; Zhang, H.; Yang, J.; Jiang, Q.; Li, C.; Yang, J.; Su, H.; et al. 2024. Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection. In *European Conference on Computer Vision*.

- Liu, Y.; Li, P.; Xie, C.; Hu, X.; Han, X.; Zhang, S.; Yang, H.; and Wu, F. 2025. Infigui-r1: Advancing multimodal gui agents from reactive actors to deliberative reasoners. *arXiv preprint arXiv:2504.14239*.
- Liu, Y.; Liu, Z.; Zhu, S.; Li, P.; Xie, C.; Wang, J.; Hu, X.; Han, X.; Yuan, J.; Wang, X.; et al. 2026. Infigui-gl: Advancing gui grounding with adaptive exploration policy optimization. In *AAAI*, 38, 32267–32275.
- Lu, Y.; Yang, J.; Shen, Y.; and Awadallah, A. 2024. Omniparser for pure vision based gui agent. *arXiv preprint arXiv:2408.00203*.
- Lu, Z.; Chai, Y.; Guo, Y.; Yin, X.; Liu, L.; Wang, H.; Xiao, H.; Ren, S.; Xiong, G.; and Li, H. 2025. UI-R1: Enhancing Efficient Action Prediction of GUI Agents by Reinforcement Learning. *arXiv preprint arXiv:2503.21620*.
- Luo, R.; Wang, L.; He, W.; Chen, L.; Li, J.; and Xia, X. 2025a. Gui-r1: A generalist r1-style vision-language action model for gui agents. *arXiv preprint arXiv:2504.10458*.
- Luo, T.; Logeswaran, L.; Johnson, J.; and Lee, H. 2025b. Visual Test-time Scaling for GUI Agent Grounding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*.
- Minderer, M.; Gritsenko, A.; Stone, A.; Neumann, M.; Weissenborn, D.; Dosovitskiy, A.; Mahendran, A.; Arnab, A.; Dehghani, M.; Shen, Z.; et al. 2022. Simple Open-Vocabulary Object Detection with Vision Transformers. In *European Conference on Computer Vision*.
- Nayak, S.; Jian, X.; Lin, K. Q.; Rodriguez, J. A.; Kalsi, M.; Awal, R.; Chapados, N.; Özsu, M. T.; Agrawal, A.; Vazquez, D.; et al. 2025. Ui-vision: A desktop-centric gui benchmark for visual perception and interaction. In *ICML*.
- Ouyang, M.; Lin, K. Q.; Shou, M. Z.; and Ng, H. T. 2026. FocusUI: Efficient UI Grounding via Position-Preserving Visual Token Selection. *arXiv preprint arXiv:2601.03928*.
- Peng, Z.; Wang, W.; Dong, L.; Hao, Y.; Huang, S.; Ma, S.; and Wei, F. 2023. Kosmos-2: Grounding Multimodal Large Language Models to the World. *arXiv preprint arXiv:2306.14824*.
- Qin, Y.; Ye, Y.; Fang, J.; Wang, H.; Liang, S.; Tian, S.; Zhang, J.; Li, J.; Li, Y.; Huang, S.; et al. 2025. UI-TARS: Pioneering automated gui interaction with native agents. *arXiv preprint arXiv:2501.12326*.
- Qwen Team. 2025a. Qwen2.5-VL technical report. *arXiv preprint arXiv:2502.13923*.
- Qwen Team. 2025b. Qwen3-VL Technical Report. *arXiv preprint arXiv:2511.21631*.
- Radford, A.; Kim, J. W.; Hallacy, C.; Ramesh, A.; Goh, G.; Agarwal, S.; Sastry, G.; Askell, A.; Mishkin, P.; Clark, J.; et al. 2021. Learning Transferable Visual Models From Natural Language Supervision. In *International Conference on Machine Learning*, 8748–8763.
- Sun, L.; Chen, X.; Chen, L.; Dai, T.; Zhu, Z.; and Yu, K. 2022. META-GUI: Towards Multi-modal Conversational Agents on Mobile GUI. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, 6699–6712. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics.
- Team, V.; Gao, C.; Gu, Z.; Liu, Y.; Qiu, X.; Shen, S.; Wen, Y.; Xia, T.; Xu, Z.; Zeng, Z.; et al. 2026. Ui-venus-1.5 technical report. *arXiv preprint arXiv:2602.09082*.
- Wang, X.; Wu, Z.; Xie, J.; Ding, Z.; Yang, B.; Li, Z.; Liu, Z.; Li, Q.; Dong, X.; Chen, Z.; et al. 2025. Mmbench-gui: Hierarchical multi-platform evaluation framework for gui agents. *arXiv preprint arXiv:2507.19478*.
- Wu, H.; Chen, H.; Cai, Y.; Liu, C.; Ye, Q.; Yang, M.-H.; and Wang, Y. 2025a. DiMo-GUI: Advancing Test-time Scaling in GUI Grounding via Modality-Aware Visual Reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, 26246–26256.
- Wu, Q.; Cheng, K.; Yang, R.; Zhang, C.; Yang, J.; Jiang, H.; Mu, J.; Peng, B.; Qiao, B.; Tan, R.; et al. 2025b. Gui-actor: Coordinate-free visual grounding for gui agents. In *NeurIPS*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2025c. Os-atlas: A foundation action model for generalist gui agents. In *ICLR*.
- Wu, Z.; Wu, Z.; Xu, F.; Wang, Y.; Sun, Q.; Jia, C.; Cheng, K.; Ding, Z.; Chen, L.; Liang, P. P.; et al. 2025d. OS-ATLAS: Foundation Action Model for Generalist GUI Agents. In *ICLR*.
- Xiao, G.; Tian, Y.; Chen, B.; Han, S.; and Lewis, M. 2024. Efficient Streaming Language Models with Attention Sinks. In *ICLR*.
- Xie, T.; Deng, J.; Li, X.; Yang, J.; Wu, H.; Chen, J.; Hu, W.; Wang, X.; Xu, Y.; Wang, Z.; et al. 2025. Scaling computer-use grounding via user interface decomposition and synthesis. In *NeurIPS*.
- Xu, H.; Zhang, X.; Liu, H.; Wang, J.; Zhu, Z.; Zhou, S.; Hu, X.; Gao, F.; Cao, J.; Wang, Z.; et al. 2026. Mobile-agent-v3.5: Multi-platform fundamental gui agents. *arXiv preprint arXiv:2602.16855*.
- Xu, H.-M.; Chen, Q.; Wang, L.; and Liu, L. 2025a. Attention-driven gui grounding: Leveraging pretrained multimodal large language models without fine-tuning. In *AAAI*, volume 39, 8851–8859.
- Xu, Y.; Wang, Z.; Wang, J.; Lu, D.; Xie, T.; Saha, A.; Sahoo, D.; Yu, T.; and Xiong, C. 2025b. Aguis: Unified pure vision agents for autonomous gui interaction. *ICML*.
- Yang, R.; Wu, Q.; Wang, Z.; Chen, H.; Yang, K.; Cheng, H.; Yao, H.; Peng, B.; Zhang, H.; Gao, J.; et al. 2026. Gui-libra: Training native gui agents to reason and act with action-aware supervision and partially verifiable rl. *arXiv preprint arXiv:2602.22190*.
- Ye, J.; Zhang, X.; Xu, H.; Liu, H.; Wang, J.; Zhu, Z.; Zheng, Z.; Gao, F.; Cao, J.; Lu, Z.; et al. 2025. Mobile-Agent-V3: Fundamental Agents for GUI Automation. *arXiv preprint arXiv:2508.15144*.
- You, H.; Zhang, H.; Gan, Z.; Du, X.; Zhang, B.; Wang, Z.; Cao, L.; Chang, S.-F.; and Yang, Y. 2024a. Ferret: Refer and Ground Anything Anywhere at Any Granularity. In *ICLR*.
- You, K.; Zhang, H.; Schoop, E.; Weers, F.; Swearngin, A.; Nichols, J.; Yang, Y.; and Gan, Z. 2024b. Ferret-UI: Grounded Mobile UI Understanding with Multimodal LLMs. In *European Conference on Computer Vision*.

Zhang, L.; Gao, L.; and Xu, M. 2025. Does chain-of-thought reasoning help mobile gui agent? an empirical study. *arXiv preprint arXiv:2503.16788*.

Zhang, L.; Xiao, Y.; Lu, X.; Cao, J.; Zhao, Y.; Zhou, J.; An, L.; Feng, Z.; Sha, W.; Shi, Y.; et al. 2026. OmegaUse: Building a General-Purpose GUI Agent for Autonomous Task Execution. *arXiv preprint arXiv:2601.20380*.

Zhou, Y.; Dai, S.; Wang, S.; Zhou, K.; Jia, Q.; and Xu, J. 2025. Gui-g1: Understanding r1-zero-like training for visual grounding in gui agents. In *NeurIPS*.

Related Work

GUI Agent Grounding

GUI grounding maps user instructions to executable screen elements or locations, serving as a core perception capability for GUI agents. Compared with general visual grounding, GUI grounding faces unique challenges from dense text, small icons, regular layouts, high-resolution screenshots, and platform-specific interaction semantics. Early GUI-oriented agents and datasets, such as META-GUI (?), connect task-oriented dialogue with real mobile GUI operations, showing that conversational agents can execute tasks on real apps without relying on task-specific backend APIs. Representative screenshot-based GUI agents and UI-centric MLLMs, such as CogAgent (?), SeeClick (?), Ferret-UI (?), and OmniParser (?), demonstrate that visual grounding can reduce reliance on HTML, DOM, or accessibility trees. Recent works further improve GUI grounding by scaling model capacity, GUI data, and action traces, including UGround (?), OS-Atlas (?), ShowUI (?), Aguvis (?), and UI-TARS (?). Meanwhile, ScreenSpot-Pro (?) shows that professional high-resolution GUI grounding remains challenging, especially for small targets, complex layouts, and fine-grained manipulation.

Most existing GUI grounding methods formulate localization as coordinate or action-parameter prediction. To improve this paradigm, recent works introduce GUI-specific reinforcement or reward signals, such as UI-R1 (?), GUI-R1 (?), InfiGUI-R1 (?), GUI-G1 (?), and InfiGUI-G1 (?). However, coordinate-based grounding may suffer from unstable numerical prediction and ambiguous point-level supervision, motivating coordinate-free, attention-based, or region-centric alternatives. Complementary to direct coordinate prediction, layout-aware GUI referring methods such as Tree-of-Lens (?) construct hierarchical layout structures from screenshots and user-indicated points, highlighting the importance of region semantics and spatial relationships for interpretable GUI understanding. Along this line, TAG (?), GUI-Actor (?), RegionFocus (?), DiMo-GUI (?), and FocusUI (?) explore intrinsic attention mining, action-region alignment, visual test-time scaling, modality-aware refinement, or position-preserving token selection. Concurrently, Re-Prefill (?) shows that the prefill stage largely determines candidate UI elements before coordinate decoding, and improves grounding via an attention-guided second prefill. This work shares our motivation that GUI grounding should not be viewed merely as autoregressive coordinate gener-

ation. However, Re-Prefill relies on raw cross-layer attention to construct target hypotheses and revisits inference with an additional prefill stage, whereas our method learns a lightweight frozen-backbone grounding head to explicitly compute token-region similarity and dynamically fuse task-relevant cues, providing a more direct, calibrated, and interpretable coordinate-free readout.

Visual Transformers Grounding

Visual Transformers provide the architectural foundation for modeling language, vision, and spatial correspondence in modern grounding systems. ViT (?) introduces patch-level visual tokenization, while CLIP (?) aligns visual and textual representations through large-scale contrastive pretraining. Building on these foundations, DETR (?), MDETR (?), GLIP (?), Grounding DINO (?), OWL-ViT (?), and SAM (?) extend detection toward phrase grounding, open-vocabulary localization, and promptable segmentation. More recently, grounded MLLMs such as Kosmos-2 (?), Shikra (?), Ferret (?), Qwen-VL (?), and Qwen2.5-VL (?) unify region understanding, coordinate prediction, and natural language generation, making grounding a general capability of multimodal instruction-following models. Despite their strong grounding ability, Transformer-based VLMs face efficiency and explainability challenges. On the efficiency side, high-resolution images introduce many visual tokens and substantially increase attention cost; therefore, efficient VLM and token compression methods such as ViLT (?), PuMer (?), FastV (?), and TokenPacker (?) reduce visual redundancy through lightweight encoders, pruning, merging, or compressed visual projectors. On the explainability side, Attention Rollout (?) and Transformer relevance propagation (?) attempt to identify visual evidence behind Transformer predictions. However, attention is not always a faithful explanation: attention sink in LLMs (?), register tokens in ViTs (?), and visual attention sink in LMMs (?) show that high-attention or high-norm tokens may serve internal computation rather than correspond to task-relevant regions. These findings suggest that efficient and explainable grounding should not simply prune or interpret tokens by raw attention scores, but should explicitly preserve spatial evidence and distinguish task-relevant visual regions from redundant or sink-like tokens.

Limitations

ZWERGE-UI operates at patch granularity, which introduces a localization ceiling on very high-resolution benchmarks (e.g., ScreenSpot-Pro) where targets may be smaller than a single patch. The patch granularity is a fundamental trade-off of the read-out approach: it avoids modifying the backbone vocabulary or training the language model head, but cannot achieve sub-patch precision without a second-stage zoom step. Additionally, our ablations cover only the LoRA-probe architecture; the CrossAttn probe configuration space (number of heads, active layers, rank) was not exhaustively searched.

Per-Layer Grounding Accuracy: Full Curves

Figure ?? shows the per-layer overlap@1 of every A7 CrossAttn probe evaluated independently (no fusion), plotted against the transformer layer index, for all four backbone models and all five benchmarks. Two training checkpoints are compared: **ckpt-800** (dashed blue, early training) and the **last checkpoint** (solid red, ckpt-3129 for Qwen2.5-VL models and ckpt-3130 for Qwen3-VL models). Horizontal dotted lines mark the corresponding fusion head accuracy.

Setup. Each probe is a CrossAttn grounding head attached to a single transformer layer; all probes are trained simultaneously with independent per-layer KL losses (§??). At evaluation, each probe predicts independently the fusion head (dotted line) combines all active probes and serves as the reference ceiling. Backbone parameters are completely frozen throughout.

Observations.

- **Inverted-U spatial profile.** All four models exhibit a consistent inverted-U shape: per-layer accuracy rises from the first probed layer, peaks around the middle of the probed range (L20–L22 for 28-layer models; L22–L25 for 36-layer models), then declines toward the final layers. This is direct evidence for the coordinate serialization bottleneck described in §?: the deepest layers progressively encode coordinate formatting rather than spatial content.
- **Training improves mid-layer probes more than shallow or deep ones.** Comparing ckpt-800 (blue) to the last checkpoint (red), the largest absolute gains appear in the middle of the probed range. Shallow probes improve less because spatial features at early layers are coarser; deep probes improve less because the layers are increasingly occupied by serialization signal that the probe cannot redirect.
- **Fusion tracks the (unknown) per-sample best layer.** The fusion dotted line stays within 1–3 pp of the peak of the per-layer curve on 16 of the 20 (model, benchmark) cells and exceeds it on the remaining 4; on ScreenSpot-Pro and ScreenSpot-v2 it falls 0.8–2.7 pp below the oracle peak. This is the intended behaviour: fusion is a single, sample-adaptive, deployable predictor that does not require per-benchmark layer selection, which the layerwise analysis (§?) shows to be non-monotonic and backbone-dependent.
- **Qwen3-VL probes cover a wider band.** GUI-Owl-1.5-8B and UI-Venus-1.5-8B (36-layer Qwen3-VL, probes at L18–L35) show a broader and flatter peak compared to the 28-layer models (probes at L14–L27). The wider effective range suggests that the spatial-to-serialization transition is more gradual in deeper networks.

Figures ??–?? show the same data broken out by benchmark, at a larger scale for readability.

Robustness, Controls, and Statistical Stability

This appendix consolidates the controls and uncertainty quantification requested by the reviewers, all computed on

	SS-Pro	SS-v2	OSWorld-G	UI-Vision	MMBench
UI-TARS-1.5-7B	20	21	21	23	20
GUI-Owl-7B	19	21	20	20	20
GUI-Owl-1.5-8B-Instruct	24	23	23	22	23
UI-Venus-1.5-8B	23	25	25	24	24

Table 10: **Spatial-peak layer L^* is stable across UI-layout distributions.** Each cell is the hit@1-peak layer on that benchmark (A8 ckpt-2800). Range is max – min across the five benchmarks.

the A8 (checkpoint-2800) probes used in the main text. All scripts and raw JSON are released with the submission.

OOD layout generalization (4KeN-W4, eKpD-W2). Table ?? reports the spatial-peak layer L^* for each (model, benchmark) cell. L^* varies by at most 3 layers within each model across the five benchmarks (std ≤ 1.1), including the distribution-shifted UI-Vision and MMBench-GUI. The inverted-U (peak > final) holds in 20/20 cells.

Bootstrap confidence intervals and seed stability (eKpD-W6). Tables ??–?? report 1000× bootstrap resamples of the $n=200$ spatial-lens set (Exp. 3) and the $n=150$ instruction-switch pairs (Exp. 4). The serialization plateau $L^\dagger$ is stable within a ≤ 1 -layer 95% CI for every model; L^* within ≤ 3 layers for three of four (≤ 7 for GUI-Owl-7B); the lag $L^\dagger - L^* > 0$ holds in 4/4 models. Absolute hit@1 carries ± 2 –4 pp 95% CI, matching the few-pp discrepancy between the $n=200$ lens curves and the full $n=1581$ per-layer summaries.

Inference-time fusion ablation (eKpD-W7, eKpD-W8). Table ?? compares four read-out strategies on the same A8 CrossAttn probes: final-layer-only, oracle best-single layer (upper bound, not deployable), static uniform averaging of active-layer predictions, and the learned CosMeta fusion. Learned fusion improves over final-only by 7–14 pp overlap@1 on SS-Pro, tracks the oracle within 1–3 pp (and exceeds it on some cells), and beats static averaging—quantitatively motivating both intermediate-layer readout and learned per-sample depth selection.

Controlled within-benchmark target-size regression (4KeN-W4, 4sok-Q2). Figure ?? and Table ?? hold the benchmark fixed (ScreenSpot-Pro, $n=1581$) and bin by target bbox area. The peak–final gap does *not* increase as targets shrink (it grows with size, floored by peak accuracy); the absolute final-layer hit@1 is $2.5\times$ worse on the smallest quintile, so the bottleneck is most damaging in absolute terms on small targets.

Pre-scale visual-token norm census (4sok-Q4; cf. Vision Transformers Need Registers). Figure ?? measures the ℓ_2 -norm of the visual-patch hidden states *before* the RMSNorm pre-scale, i.e. the magnitude information the pre-scale discards. The mean norm grows $\sim 3.4\times$ from layer 14 to layer 26 in GUI-Owl-7B (Qwen2.5-VL), directly confirming the depth-growth claim of §??; a small ($\leq 0.6\%$) but heavy-tailed set of outlier patches carries very high

Model
UI-TARS-
GUI-Owl-
GUI-Owl-
UI-Venus-

Table 12: **Peak layer L^* is stable across benchmarks.** $n=150$. For each benchmark, the peak layer is the layer with the highest hit@1 (L23–25)–

A7 CrossAttn Probe: Per-Layer overlap@1 (dashed=ckpt-800, solid=last ckpt)
Dotted horizontal lines show the fusion head accuracy

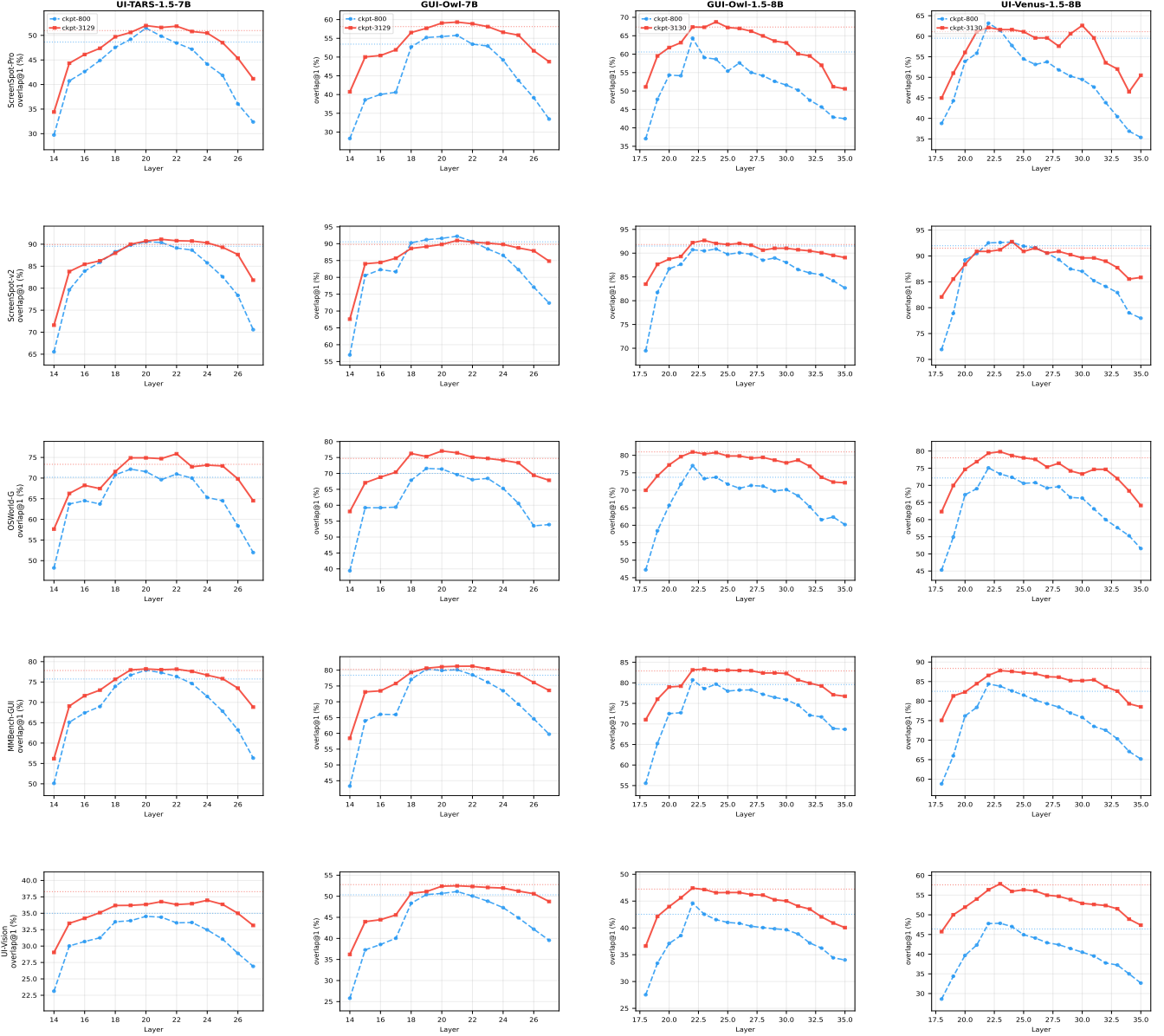


Figure 7: Per-layer overlap@1 of A7 CrossAttn probes across all four backbone models and five benchmarks. Each cell shows overlap@1 for a single probe layer evaluated independently (no cross-layer fusion). **Rows:** ScreenSpot-Pro (SS-Pro), ScreenSpot-v2 (SS-v2), OSWorld-G, MMBench-GUI, UI-Vision. **Columns:** UI-TARS-1.5-7B, GUI-Owl-7B, GUI-Owl-1.5-8B, UI-Venus-1.5-8B. **Blue dashed:** ckpt-800 (early training); **red solid:** last checkpoint (ckpt-3129 / ckpt-3130). **Horizontal dotted lines:** corresponding fusion head accuracy (the final ZWERGE-UI output). All four models exhibit an inverted-U spatial profile: accuracy peaks at intermediate layers and declines at the deepest layers, consistent with the coordinate serialization bottleneck (§??). The fusion head (dotted) tracks the best single-layer probe within 1–3 pp across nearly all panels (and exceeds it on a few), providing a deployable, sample-adaptive readout that does not require per-benchmark layer selection.

Per-Layer overlap@1 on ScreenSpot-Pro
(A7 CrossAttn Probe, each probe layer evaluated independently)

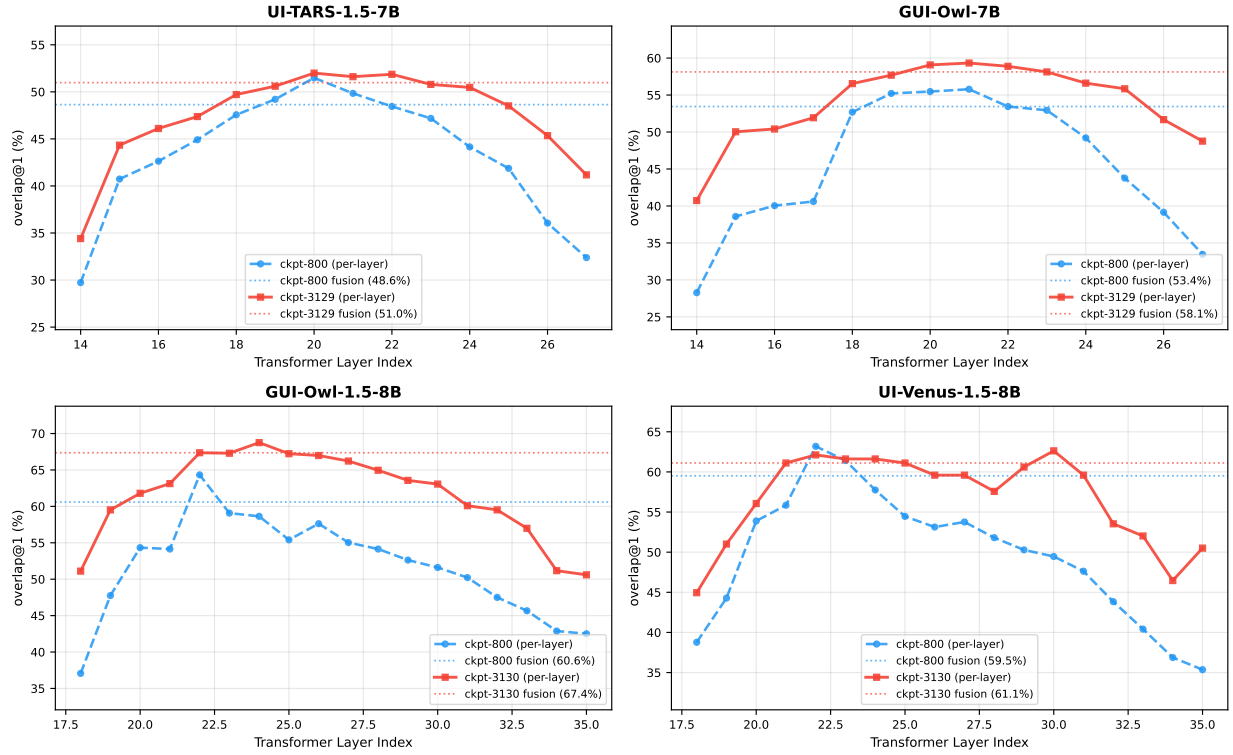


Figure 8: **Per-layer overlap@1 on ScreenSpot-Pro**. ScreenSpot-Pro is the most demanding benchmark (fine-grained targets on high-resolution desktop screenshots). The inverted-U profile is the sharpest here, with the deepest layers showing the steepest decline, reflecting that high-resolution targets require the most precise spatial encoding and are most sensitive to the serialization shift in deep layers. Note that the vertical scales differ across panels to emphasize the shape of the curve within each model.

Per-Layer overlap@1 on ScreenSpot-v2
(A7 CrossAttn Probe, each probe layer evaluated independently)

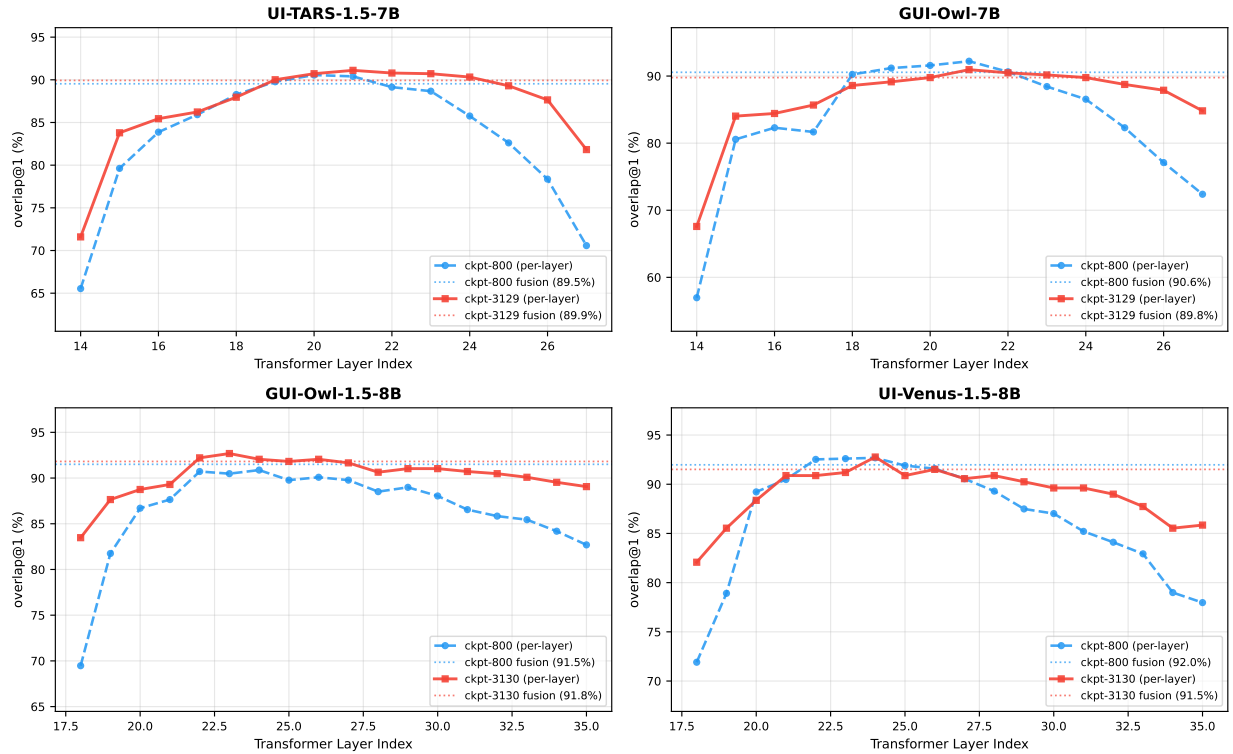


Figure 9: **Per-layer overlap@1 on ScreenSpot-v2.** ScreenSpot-v2 is a moderate-resolution benchmark (mobile, desktop, web) where overall accuracy is already high ($>87\%$ at ckpt-800). The inverted-U is shallower and the peak is broader compared to ScreenSpot-Pro, indicating that moderate-resolution grounding is less sensitive to which specific layer is read.

Per-Layer overlap@1 on OSWorld-G
(A7 CrossAttn Probe, each probe layer evaluated independently)

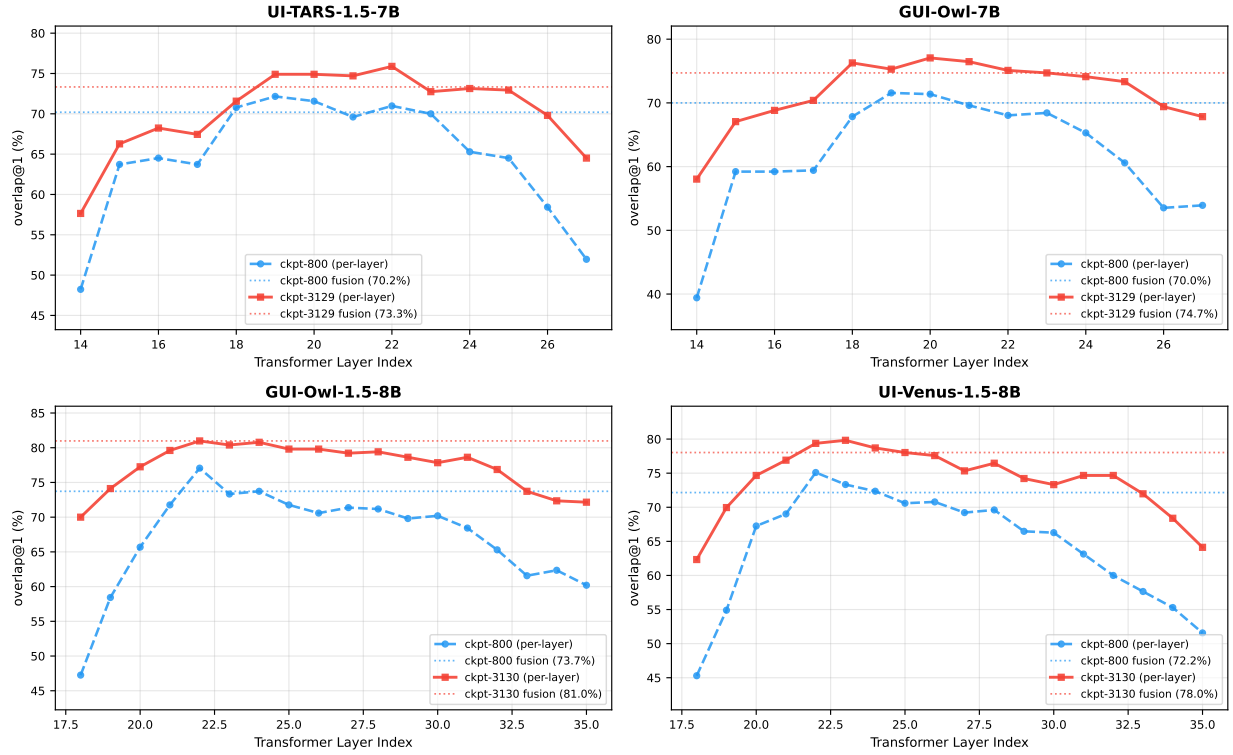


Figure 10: **Per-layer overlap@1 on OSWorld-G.** OSWorld-G provides real computer-task GUI samples with diverse task types (Text Matching, Element Recognition, Layout Understanding, Fine-grained Manipulation). The per-layer profile shows a similar inverted-U shape but with a plateau over a wider middle range ($\sim$ L18–L24 for 28-layer models), suggesting that layout-level spatial reasoning is distributed more broadly across intermediate layers than fine-grained element targeting.

Per-Layer overlap@1 on MMBench-GUI
(A7 CrossAttn Probe, each probe layer evaluated independently)

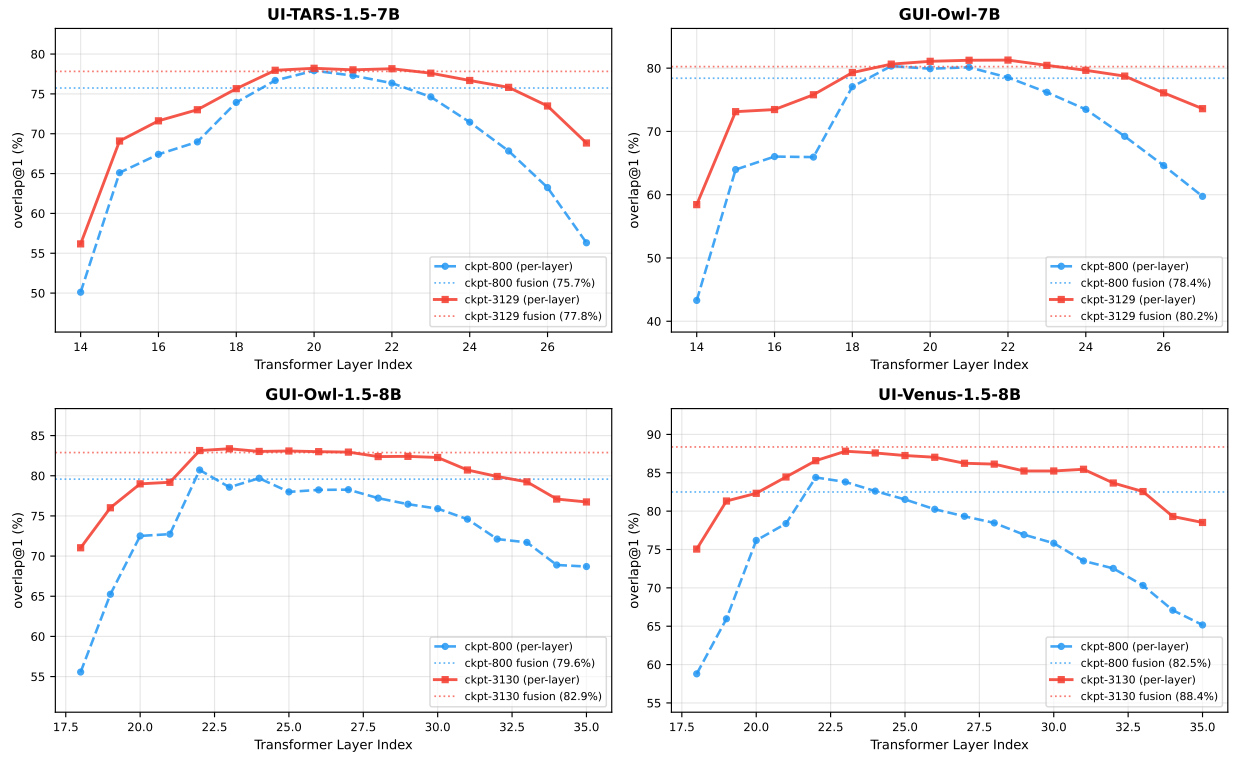


Figure 11: **Per-layer overlap@1 on MMBench-GUI**. MMBench-GUI covers six platforms (Windows, macOS, Linux, iOS, Android, Web). The per-layer curves are notably high even at ckpt-800 (blue dashed) for Qwen3-VL models (GUI-Owl-1.5-8B and UI-Venus-1.5-8B), reflecting that these models have strong cross-platform generalization already encoded in their intermediate representations. The gap between ckpt-800 and last checkpoint is smaller on MMBench-GUI than on ScreenSpot-Pro, suggesting that the per-layer signal saturates earlier on this benchmark.

Per-Layer overlap@1 on UI-Vision
(A7 CrossAttn Probe, each probe layer evaluated independently)

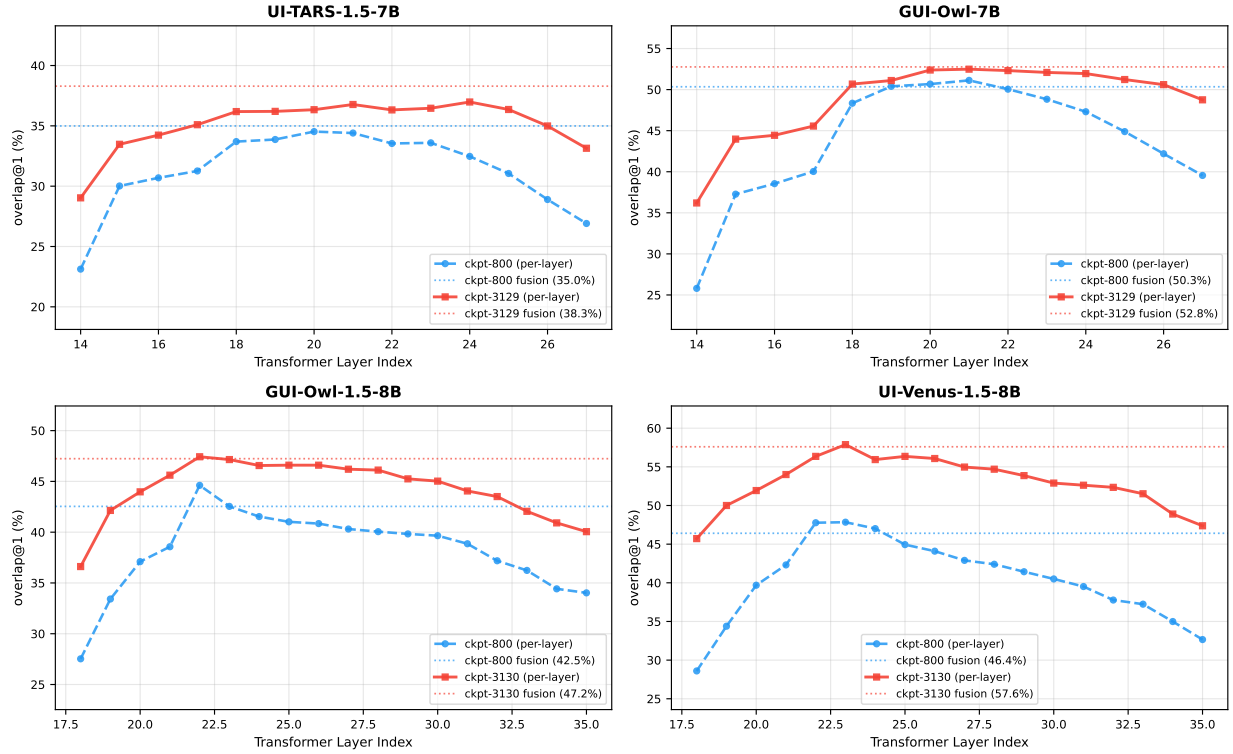


Figure 12: **Per-layer overlap@1 on UI-Vision.** UI-Vision decomposes grounding into Basic, Functional, Spatial, and Layout subtypes; accuracy here represents the weighted average over all 5,790 samples. Per-layer accuracy is substantially lower on UI-Vision than on other benchmarks, particularly for Qwen2.5-VL models at early training (ckpt-800 for UI-TARS-1.5-7B peaks below 30%), indicating that UI-Vision’s diverse task composition is harder for intermediate probes to solve with limited training. The inverted-U is still present but shallower and shifted earlier (peak around L18–L20), possibly because UI-Vision includes layout-level tasks that require higher-level semantic understanding.

Prompt for the UI-TARS-1.5 Model

1 You are a helpful assistant.

2

3 You are a GUI agent. You are given a task and your action history, with screenshots.

4 You need to perform the next action to complete the task.

5

6 ## Output Format

7 Action: ...

8

9 ## Action Space

10 click(start_box='<|box_start|>(x1,y1)<|box_end|>')

11

12 ## User Instruction

13 Task Instruction: {instruction}

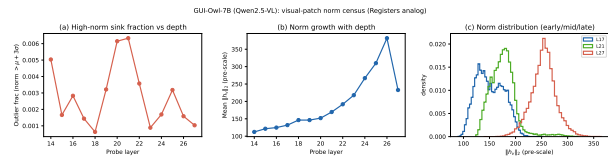


Figure 18: **Pre-scale visual-patch norm census (GUI-Owl-7B, $n=100$).** (a) high-norm sink fraction vs depth; (b) mean $\|h_k\|_2$ grows $\sim 3.4\times$ across depth; (c) norm distribution at early/mid/late layers (heavy-tailed).

norm, consistent with the attention-sink / register-token phenomenon—information the pre-scale normalizes away but which a magnitude-aware variant could exploit.

Prompt for the GUI-Owl Model

```
1  You are a helpful assistant.

2  # Tools

3  You may call one or more functions to assist with the user query.

4  You are provided with function signatures within <tools></tools> XML tags:

5  <tools>

6  {

7      "type": "function",

8      "function": {

9          "name": "computer_use",

10         "description": "Use a mouse to interact with a computer screen."

11         "* The screen's resolution is ${height}x${width}."

12         "* Make sure to click any buttons, links, icons, etc with the cursor tip in the center of the
           element. Don't click boxes on their edges unless asked.",

13         "parameters": {

14             "properties": {

15                 "action": {

16                     "description": "The action to perform. The available actions are:"

17                     "* 'left_click': Click the left mouse button at a specified (x, y) pixel coordinate
                       on the screen.",

18                     "enum": ["left_click"],

19                     "type": "string"

20                 },

21                 "coordinate": {

22                     "description": "(x, y): The x (pixels from the left edge) and y (pixels from the top edge)
                       coordinates to click.",

23                     "type": "array"
```

Prompt for the GUI-Owl-1.5 Model

1 # Tools

2

3 You may call one or more functions to assist with the user query.

4

5 You are provided with function signatures within <tools></tools> XML tags:

6 <tools>

7 {"type": "function", "function": {"name": "computer_use", "description": "Use a mouse to interact with a computer.\n* The screen's resolution is 1000x1000.\n* Make sure to click any buttons, links, icons, etc

with the cursor tip in the center of the element. Don't click boxes on their edges unless asked.", "

parameters": {"properties": {"action": {"description": "The action to perform. The available actions are

: \n* 'left_click': Click the left mouse button with coordinate (x, y) pixel coordinate on the screen.",

"enum": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the

left edge) and y (pixels from the top edge) coordinates to click.", "type": "array"}}, "required": ["

action"], "type": "object"}}}

8 </tools>

9

10 For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:

11 <tool_call>

12 {"name": <function-name>, "arguments": <args-json-object>}

13 </tool_call>

14

15 ## User Instruction

16 {instruction}

Prompt for the UI-Venus-1.5 Model

1 Output the center point of the position corresponding to the following instruction: {instruction}.

2

3 The output should just be the coordinates of a point, in the format [x,y].

Prompt for the UI-TARS-1.5 Retrofit Model

```
1  [System]

2  You are a GUI agent. You are given a task and a screenshot.

3  You need to perform the next action to complete the task.

4

5  ## Output Format

6

7  Action: ...

8

9  ## Action Space

10

11 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')

12

13 [User]

14 <image>

15 {instruction}

16

17 [Assistant]

18 click(start_box='<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>')
```

Prompt for the GUI-Owl-7B / GUI-Owl-1.5 Retrofit Model

```
1  [System]

2  # Tools

3

4  You may call one or more functions to assist with the user query.

5

6  You are provided with function signatures within <tools></tools> XML tags:

7  <tools>

8  {"type": "function", "function": {

9    "name": "computer_use",

10   "description": "Use a mouse and keyboard to interact with a computer, and take screenshots. This is an
      interface to a desktop GUI. You do not have access to a terminal or applications menu. You must click on

      desktop icons to start applications. * The screen's resolution is 1000x1000.\n* Make sure to click

      buttons, links, icons, etc with the cursor tip in the center of the element. Don't click boxes on their

      edges unless asked.",

11   "parameters": {"properties": {"action": {"description": "The action to perform. The available actions are:\n*
      'left_click': Click the left mouse button at coordinate (x, y) pixel coordinate on the screen.", "enum

      ": ["left_click"], "type": "string"}, "coordinate": {"description": "(x, y): The x (pixels from the left

      edge) and y (pixels from the top edge) coordinates to click. Required only by 'action=left_click'."}, "

      type": "array"}}, "required": ["action", "coordinate"], "type": "object"}}}

12 </tools>

13

14 For each function call, return a json object with function name and arguments within <tool_call></tool_call>
    XML tags:

15 <tool_call>

16 {"name": "computer_use", "arguments": {"action": "left_click", "coordinate": [<|ground|><|pointer_start|><|
    pointer_pad|><|pointer_end|>]}}

17 </tool_call>
```

Prompt for the UI-Venus-1.5 Retrofit Model

1 [User]

2 <image>

3 Output the center point of the position corresponding to the following instruction:

4 {instruction}.

5

6 The output should just be the coordinates of a point, in the format

7 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>].

8

9 [Assistant]

10 [<|ground|><|pointer_start|><|pointer_pad|><|pointer_end|>]



